FC7xxx ADC Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0

Table of Contents

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the ADC module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- MCU: This module provides the clock reference point for ADC module.
- Port: This module provides pin port configuration to be used as ADC channels.
- Common: This module is the basic module which used to choose the chip variant.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the ADC module, the user needs to initialize MCU module first.
- Port: To use ADC external channels, the user needs to configure the physical ports connected to the Adc channels.

2.2 Files Required for Compile

ADC module files:

- _MCAL/Src/Adc/Src/Adc.c
- _MCAL/Src/Adc/Src/Adc_Hw.c
- _MCAL/Src/Adc/Src/Adc_Lld.c
- _MCAL/Src/Adc/Src/Adc_Irq.c
- _MCAL/Src/Adc/Src/Adc_Ptimer_Hw.c
- _MCAL/Src/Adc/Src/Adc_Ptimer_Irq.c
- _MCAL/Src/Adc/include/Adc.h
- _MCAL/Src/Adc/include/Adc_Hw.h
- _MCAL/Src/Adc/include/Adc_Lld.h
- _MCAL/Src/Adc/include/Adc_RegOps.h
- _MCAL/Src/Adc/include/Adc_Reg.h
- _MCAL/Src/Adc/include/Adc_Ptimer_Hw.h
- _MCAL/Src/Adc/include/Adc_Ptimer_RegOps.h
- _MCAL/Src/Adc/include/Adc_Ptimer_Reg.h
- _MCAL/Src/Adc/include/Adc_Types.h
- _MCAL/Src/Adc/include/Adc_Hw_Types.h
- _MCAL/Src/Adc/include/Adc_Ptimer_Hw_Types.h
- _MCAL/Src/Adc/include/Adc_MemMap.h
- _MCAL/Src/Adc/include/Adc_Version.h

- _MCAL_generate/src/Adc_PBcfg.c
- _MCAL_generate/src/Adc_Cfg.c
- _MCAL_generate/include/Adc_Cfg.h

- _MCAL_generate/include/Adc_CfgDefines.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Adc.h

2.3 Add Plug-ins

ADC module plug-ins are developed for EB tresos Studio, so, to use ADC plug-ins on the EB tresos Studio, the user needs to add the ADC module to the EB plug-ins folder first.

1) Copy the ADC module(_MCAL/EB_Plugins/eclipse/plugins/Adc) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
2) Set the ADC module output location folder for the generated source file and header files.
3) Use EB tresos to configure the ADC module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
ADC_START_SEC_CONFIG_DATA_8 ADC_STOP_SEC_CONFIG_DATA_8 ADC_START_SEC_CONFIG_DATA_16 ADC_STOP_SEC_CONFIG_DATA_16 ADC_START_SEC_CONFIG_DATA_32 ADC_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
ADC_START_SEC_CONFIG_DATA_UNSPECIFIED ADC_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
ADC_START_SEC_CONST_BOOLEAN ADC_STOP_SEC_CONST_BOOLEAN ADC_START_SEC_CONST_8 ADC_STOP_SEC_CONST_8 ADC_START_SEC_CONST_16 ADC_STOP_SEC_CONST_16 ADC_START_SEC_CONST_32 ADC_STOP_SEC_CONST_32 ADC_START_SEC_CONST_ UNSPECIFIED ADC_STOP_SEC_CONST_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
ADC_START_SEC_CODE ADC_STOP_SEC_CODE ADC_START_SEC_ RAMCODE ADC_STOP_SEC_ RAMCODE ADC_START_SEC_CODE_AC ADC_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
ADC_START_SEC_VAR_NO_INIT_BOOLEAN ADC_STOP_SEC_VAR_NO_INIT_BOOLEAN ADC_START_SEC_VAR_NO_INIT_8 ADC_STOP_SEC_VAR_NO_INIT_8 ADC_START_SEC_VAR_NO_INIT_16 ADC_STOP_SEC_VAR_NO_INIT_16 ADC_START_SEC_VAR_NO_INIT_32 ADC_STOP_SEC_VAR_NO_INIT_32 ADC_START_SEC_VAR_NO_INIT_ UNSPECIFIED ADC_STOP_SEC_VAR_NO_INIT_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
ADC_START_SEC_VAR_ INIT_BOOLEAN ADC_STOP_SEC_VAR_ INIT_BOOLEAN ADC_START_SEC_VAR_ INIT_8 ADC_STOP_SEC_VAR_ INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
ADC_START_SEC_VAR _INIT_16 ADC_STOP_SEC_VAR _INIT_16 ADC_START_SEC_VAR _INIT_32 ADC_STOP_SEC_VAR _INIT_32 ADC_START_SEC_VAR _INIT_ UNSPECIFIED ADC_STOP_SEC_VAR _INIT_ UNSPECIFIED		startup code (data).

Chapter 4 Exclusive Area

ADC module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the Adc driver:

- Adc.c:
  - Adc_DeInit : exclusive area 0
  - Adc_StartGroupConversion : exclusive area 1
  - Adc_StopGroupConversion : exclusive area 2
  - Adc_ReadGroup : exclusive area 3
  - Adc_EnableHardwareTrigger : exclusive area 4
  - Adc_DisableHardwareTrigger : exclusive area 5
  - Adc_GetGroupStatus : exclusive area 6
  - Adc_GetStreamLastPointer : exclusive area 7

- Adc_Hw.c:
  - Adc_HL_RemoveFromQueue : exclusive area 8
  - Adc_HL_ReadResultBuffer : exclusive area 9
  - Adc_HL_UpdateSwQueue : exclusive area10, 11
  - Adc_HL_UpdateStatusGetData : exclusive area 12
  - Adc_HL_UpdateStatusReadGroup : exclusive area 13
  - Adc_HL_StartConversion : exclusive area 14, 15, 16
  - Adc_HL_StopConversion : exclusive area 17, 18
  - Adc_HL_EnableHardwareTrigger : exclusive area 19
  - Adc_HL_DisableHardwareTrigger : exclusive area 20
  - Adc_HL_EndPartialConversion : exclusive area 21, 22

- Adc_Lld.c:
  - Adc_LL_ConfigurePrescaler : exclusive area 23, 24
  - Adc_LL_StopConversionCheckTimeout : exclusive area 25
  - Adc_LL_DisableUnitCheckTimeout : exclusive area 26
  - Adc_LL_ConfigureDmaChannel : exclusive area 27
  - Adc_LL_InitUnitHardware : exclusive area 28, 29, 30

- Adc_LL_DeInitUnitHardware : exclusive area 31
- Adc_LL_StartHwTrigConversion : exclusive area 32
- Adc_LL_EnableHardwareTrigger : exclusive area 33
- Adc_LL_DisableHardwareTrigger : exclusive area 34
- Adc_LL_StartNormalConversion : exclusive area 35
- Adc_LL_ClearConvCompleteFlag : exclusive area 36
- Adc_LL_ConfigurePartialConversion : exclusive area 37
- Adc_LL_StopCurrentConversion : exclusive area 38
- Adc_LL_RestartContinuousConversion : exclusive area 39
- Adc_LL_CheckConversionSequenceStatus : exclusive area 40
- Adc_LL_GetConversionSequenceResults : exclusive area 41
- Adc_LL_ReConfigureDma : exclusive area 42

- Adc_Ptimer_Hw.c:
  - Adc_Ptimer_InitUnitHardware : exclusive area 43
  - Adc_Ptimer_DeInitUnitHardware : exclusive area 44
  - Adc_Ptimer_ConfigurePartialConversion : exclusive area 45
  - Adc_Ptimer_StartSoftwareConversion : exclusive area 46
  - Adc_Ptimer_SetPtimerMode : exclusive area 47
  - Adc_Ptimer_StopConversion : exclusive area 48

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
ADC0	ADC0_IRQHandler	131
ADC1	ADC1_IRQHandler	132
ADC2	ADC2_IRQHandler	133
ADC3	ADC3_IRQHandler	134

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Adc_Init	ADC_E_ALREADY_INITIALIZED; ADC_E_PARAM_CONFIG
Adc_SetupResultBuffer	ADC_E_UNINIT; ADC_E_PARAM_POINTER; ADC_E_PARAM_GROUP; ADC_E_BUSY
Adc_DeInit	ADC_E_UNINIT; ADC_E_BUSY
Adc_StartGroupConversion	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_WRONG_TRIGG_SRC; ADC_E_BUFFER_UNINIT; ADC_E_BUSY; ADC_E_QUEUE_FULL
Adc_StopGroupConversion	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_WRONG_TRIGG_SRC; ADC_E_IDLE
Adc_ReadGroup	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_IDLE
Adc_EnableHardwareTrigger	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_BUFFER_UNINIT; ADC_E_WRONG_TRIGG_SRC; ADC_E_WRONG_CONV_MODE; ADC_E_BUSY
Adc_DisableHardwareTrigger	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_WRONG_TRIGG_SRC; ADC_E_WRONG_CONV_MODE; ADC_E_IDLE
Adc_EnableGroupNotification	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_NOTIF_CAPABILITY
Adc_DisableGroupNotification	ADC_E_UNINIT; ADC_E_PARAM_GROUP;

Function Name	Error Type
	ADC_E_NOTIF_CAPABILITY
Adc_GetGroupStatus	ADC_E_UNINIT; ADC_E_PARAM_GROUP
Adc_GetStreamLastPointer	ADC_E_UNINIT; ADC_E_PARAM_GROUP; ADC_E_IDLE
Adc_GetVersionInfo	ADC_E_PARAM_POINTER

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs be called is Adc_Init(const Adc_ConfigType *ConfigPtr) and Adc_SetupResultBuffer(Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr).

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

  If the user configures notifications and the value is not "NULL_PTR" or "NULL", the user shall implement the notification function in the user code.

- Callback

  None

- Callout

  None

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

  AUTOSAR_OS_NOT_USED need defined.

  In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example：

```
extern ISR(Adc_ISR_EndGroupConvUnit0);
void ADC0_IRQHandler(void)
{
  Adc_ISR_EndGroupConvUnit0();
}
```

- If OS is used:

  Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

1) Configure the ADC module and generate configuration files (please refer to Building chapter for details).
2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
4) Build the ADC module with dependent modules.